

Validación de Suficiencia por Competencias**RESULTADO DE APRENDIZAJE**
Fase 3 estructuras repetitivas y arreglos

PRESENTADO POR:
Anderson Javier Hurtado Moreno
CÓDIGO: 1013630063
GRUPO:213022B_2201

DOCENTE:
JULIAN ANDRES RUIZ

UNIVERSIDAD NACIONAL ABIERTA Y A DISTANCIA - UNAD
ESCUELA DE CIENCIAS BÁSICAS, INGENIERÍAS Y TECNOLOGÍAS
CURSO DE FUNDAMENTOS DE PROGRAMACION COD:213022B
14 DE ABRIL DE 2026
LUGAR: BOGOTA
2026

Introducción

Las estructuras repetitivas y los arreglos son fundamentales en programación, ya que permiten manejar múltiples datos de manera eficiente. Las estructuras repetitivas, como `for` y `while`, se utilizan para ejecutar instrucciones varias veces, mientras que los arreglos permiten almacenar y organizar información en una sola variable.

La combinación de estas herramientas facilita el desarrollo de programas más ordenados y funcionales, permitiendo procesar datos como listas de productos, precios o cantidades de forma rápida y sencilla.

Problema 1,**Tabla de requerimientos**

Identificación del requerimiento	Descripción	Entradas	Resultados o salidas
R1	Solicitar al usuario una temperatura positiva entera en °c	Valor de temperatura ingresado por el usuario	Temperatura almacenada en la variable "temperatura"
R2	Validar que la temperatura ingresada sea un numero entero positivo	Temperatura ingresada	Confirmación de dato valido o mensaje de error
R3	Evaluar si la temperatura es mayor a 200°C	Variable temperatura	Mensaje alerta de enfriamiento
R4	Evaluar si la temperatura es menor a 180°C	Variable temperatura	Mensaje alerta de calentamiento
R5	Evaluar si la temperatura esta dentro del rango 180°C-200°C	Variable temperatura	Mensaje de temperatura optima
R6	Preguntar al usuario si desea continuar	Opción del usuario 1:si, 0:no	Continuar el programa o finalizar el monitoreo

Código fuente:

```
def pedir_temperatura():  
  
    while True:  
  
        try:  
  
            temp = int(input("Ingrese la temperatura en °C: "))  
  
            if temp < 0:  
  
                print("Error: la temperatura debe ser un número positivo.\n")  
  
            else:  
  
                return temp  
  
        except:  
  
            print("Error: debe ingresar un número entero.\n")  
  
  
def evaluar_temperatura(temp):  
  
    if temp > 200:  
  
        print("● Alerta: enfriamiento\n")  
  
    elif temp < 180:  
  
        print("□ Alerta: calentamiento\n")  
  
    else:  
  
        print("□ Temperatura óptima\n")  
  
  
def preguntar_continuar():  
  
    while True:  
  
        try:  
  
            opcion = int(input("¿Desea continuar? (1: Sí / 0: No): "))
```

```
if opcion in (0, 1):
```

```
    return opcion
```

```
else:
```

```
    print("Error: ingrese 1 o 0.\n")
```

```
except:
```

```
    print("Error: debe ingresar un número.\n")
```

```
continuar = 1
```

```
while continuar == 1:
```

```
    temperatura = pedir_temperatura()
```

```
    evaluar_temperatura(temperatura)
```

```
    continuar = preguntar_continuar()
```

```
print("\nMonitoreo finalizado. ¡Gracias!")
```

Problema 6,**Tabla de requerimientos**

Identificación del requerimiento	Descripción	Entradas	Resultados o salidas
R1	Solicitar cantidad de cliente	Numero de clientes	Inicio del proceso
R2	Mostrar menú de pizzas desde la matriz	Dato de la matriz	Menú en la pantalla
R3	Permitir seleccionar pizzas y finalizar con 0	Tipo de pizza (1-50 ° 0)	Registro de pedido
R4	Validar opción ingresada	Numero ingresado	Mensaje de error
R5	Calcular totales por cliente y porciones	Selecciones	Totales acumulados
R6	Mostrar resumen final de ventas	Datos acumulados	Cantidad vendida, total ventas y porciones

Código fuente:

```
#areglo matriz

pizzas = [

    ["Pequeña", 2, 10000],

    ["Mediana", 4, 15000],

    ["Larga", 6, 20000],

    ["Familiar", 8, 25000],

    ["Extra Familiar", 12, 35000]

]


cantidad_vendida = [0, 0, 0, 0, 0]

valor_total_ventas = 0

total_porciones = 0


num_clientes = int(input("Ingrese la cantidad de clientes: "))


for cliente in range(num_clientes):

    print("\nCliente", cliente + 1)

    total_cliente = 0


    tipo = -1
```

```
while tipo != 0:

    print("\nMenu de pizzas")

    for i in range(5):

        print(i + 1, pizzas[i][0], "-", pizzas[i][1], "porciones - $",
pizzas[i][2])

    print("0. Finalizar orden")


tipo = int(input("Seleccione el tipo de pizza: "))


if 1 <= tipo <= 5:

    indice = tipo - 1

    cantidad_vendida[indice] += 1

    total_cliente += pizzas[indice][2]

    total_porciones += pizzas[indice][1]

    print("Agregada:", pizzas[indice][0])


elif tipo == 0:

    print("Orden finalizada")

else:

    print("Opcion no valida")


valor_total_ventas += total_cliente

print("Total del cliente: $", total_cliente)


print("\nResumen final")

for i in range(5):
```



```
print(pizzas[i][0], ":", cantidad_vendida[i], "vendidas")
```

```
print("Valor total de ventas: $", valor_total_ventas)
```

```
print("Total de porciones vendidas:", total_porciones)
```

Conclusiones

En conclusión, las estructuras repetitivas y los arreglos permiten desarrollar programas más eficientes y organizados. Gracias a ellas, es posible manejar múltiples datos, automatizar procesos y obtener resultados de forma rápida. Su uso es fundamental para resolver problemas como el control de ventas, facilitando el registro y cálculo de información de manera sencilla

Bibliografía

- Rocio Rodríguez Guerrero, Carlos Alberto Vanegas, & Gerardo Castang Montiel. (2020). [Python a su alcance. Universidad Distrital Francisco José deCaldas. https://research-ebsco-com.bibliotecavirtual.unad.edu.co/c/qcagk4/ebook-viewer/epub/kxivjenv2z/section/a5](https://research-ebsco-com.bibliotecavirtual.unad.edu.co/c/qcagk4/ebook-viewer/epub/kxivjenv2z/section/a5)
- Van Rossum, G., & Drake Jr, F. L. (2024). [El tutorial de Python. Python Software Foundation. https://docs.python.org/es/3.12/tutorial/index.htm](https://docs.python.org/es/3.12/tutorial/index.htm)
|